Lightweight Real-Time Intrusion Detection for IoT Networks Using TSANet Deployed on Raspberry Pi with Live Traffic Monitoring

(Author information removed for double-blind review)

Abstract

Intrusion Detection Systems (IDS) are crucial for securing IoT networks, but many deep learning models are too heavy for deployment on resource-constrained edge devices. This work presents TSANet, a lightweight Temporal–Spatial Attention Network designed for real-time intrusion detection on low-power hardware such as the Raspberry Pi 5. TSANet integrates a 1D CNN for spatial feature extraction, an LSTM for temporal modeling, and dual attention mechanisms to highlight critical features and time steps. Trained on the CICIDS2017 dataset using 32 engineered flow features, the model is optimized through pruning and quantization-aware training. TSANet achieves 98.3% offline accuracy and is deployed on a Raspberry Pi configured as a live Wi-Fi access point. Real-time tests show 96.8% detection accuracy with 8–12 ms latency per flow. These results demonstrate that compressed attention-based models can enable practical, low-latency IDS on IoT edge devices.

Keywords— Intrusion Detection System, IoT Security, Raspberry Pi, Temporal–Spatial Attention, Deep Learning, Edge Computing, TSANet, Traffic Classification.

1 Introduction

The motivation for this work originated from an observation made during the analysis of modern smart city infrastructure projects, where Internet-of-Things (IoT) devices form the backbone of automation, monitoring, and control. While exploring the challenges faced by edge systems in real-time network communication, it became evident that security was a growing concern, particularly at the device level. The increasing dependence on IoT networks in sectors such as energy, transportation, and public safety revealed that even a minor intrusion could disrupt critical services. This insight led to the identification of a fundamental problem: existing Intrusion Detection Systems (IDS) are not designed for lightweight, real-time deployment on constrained IoT devices such as Raspberry Pi.

The problem statement was formally developed after reviewing multiple research studies and observing network behavior in simulated IoT environments. It became apparent that centralized IDS architectures, though powerful, are impractical for continuous edge operation due to their computational and latency overhead. These systems require large-scale data transfer to cloud servers for analysis, which introduces bottlenecks and delays. This motivated the exploration of localized, on-device intrusion detection mechanisms that could efficiently detect attacks such as Denial of Service (DoS), Distributed Denial of Service (DDoS), brute force, port scanning, and protocol-specific exploits like Heartbleed, all in real time.

The research direction was refined through consultations with cybersecurity experts and mentors at KLE Technological University. The guidance and feedback received helped shape the prob-

lem statement into a focused objective: to design a deep learning–based intrusion detection system optimized for low-power, resource-constrained edge devices. This formulation directly aligns with practical deployment scenarios found in academic IoT laboratories and industrial smart systems, where real-time detection and energy efficiency are equally important.

The problem domain also intersects with global research efforts in IoT security. Organizations such as the *International Telecommunication Union (ITU)* and the *United Nations Development Programme (UNDP)* have identified network resilience and secure digital infrastructure as key components of sustainable development. In particular, the proposed work aligns with the United Nations **Sustainable Development Goal 9 (Industry, Innovation, and Infrastructure)**, which emphasizes the development of reliable, sustainable, and resilient infrastructure to support economic growth and human well-being [1]. This global context highlights the broader impact of the study beyond its technical contributions. https://sdgs.un.org/goals/goal9

To address these challenges, this research proposes TSANet — a Temporal–Spatial Attention Network that integrates convolutional neural networks (CNNs), spatial attention mechanisms, and long short-term memory (LSTM) layers enhanced with temporal attention. TSANet is designed to simultaneously capture spatial dependencies among network flow features and temporal dependencies across traffic sequences. Furthermore, the model is optimized through pruning and quantization-aware training to ensure lightweight deployment on Raspberry Pi hardware for real-time intrusion detection.

The main contributions of this study can be summarized as follows: (i) design of a hybrid CNN–Attention–LSTM architecture for IoT intrusion detection, (ii) feature engineering using 32 flow features from the CICIDS-2017 dataset, (iii) model optimization through pruning and quantization-aware training, (iv) deployment of the optimized TSANet model on a Raspberry Pi configured as a wireless access-point router for real-time traffic monitoring and attack detection, and (v) evaluation of real-time performance using live wireless traffic, flow-level analysis, and physical alert mechanisms. Collectively, this work demonstrates that attention-enhanced deep learning can enable reliable, low-latency intrusion detection at the IoT edge, directly supporting the vision of secure and sustainable smart infrastructures.

Figure 1 presents a conceptual overview of the motivation and development flow that shaped the proposed TSANet-based Intrusion Detection System for IoT environments. The upper-left portion of the diagram highlights the foundation of the work, beginning with the increasing adoption of *Smart City Infrastructure* and *IoT Networks*, which introduce new opportunities for automation but simultaneously expand the attack surface. This rising dependency forms the core *Motivation* for advancing lightweight, real-time intrusion detection mechanisms.

The next section of the diagram illustrates the limitations of traditional security approaches. *Centralized Architectures* and their significant *Computational Overhead* are shown as key challenges that make conventional IDS solutions unsuitable for constrained IoT devices, thereby defining the *Problem Statement*. Moving downward, the diagram emphasizes how ongoing advancements in *Deep Learning* and the need for models compatible with *Resource-Constrained Devices* shaped the *Research Direction* of this study.

The *Global Context* branch references contributions from organizations such as the ITU and UNDP, linking this research to broader efforts supporting secure and sustainable digital infrastructure. At the center, the diagram converges into the proposed solution—*TSANet: Deep Learning for IoT Intrusion Detection*—which integrates spatial and temporal attention mechanisms tailored for edge deployment.

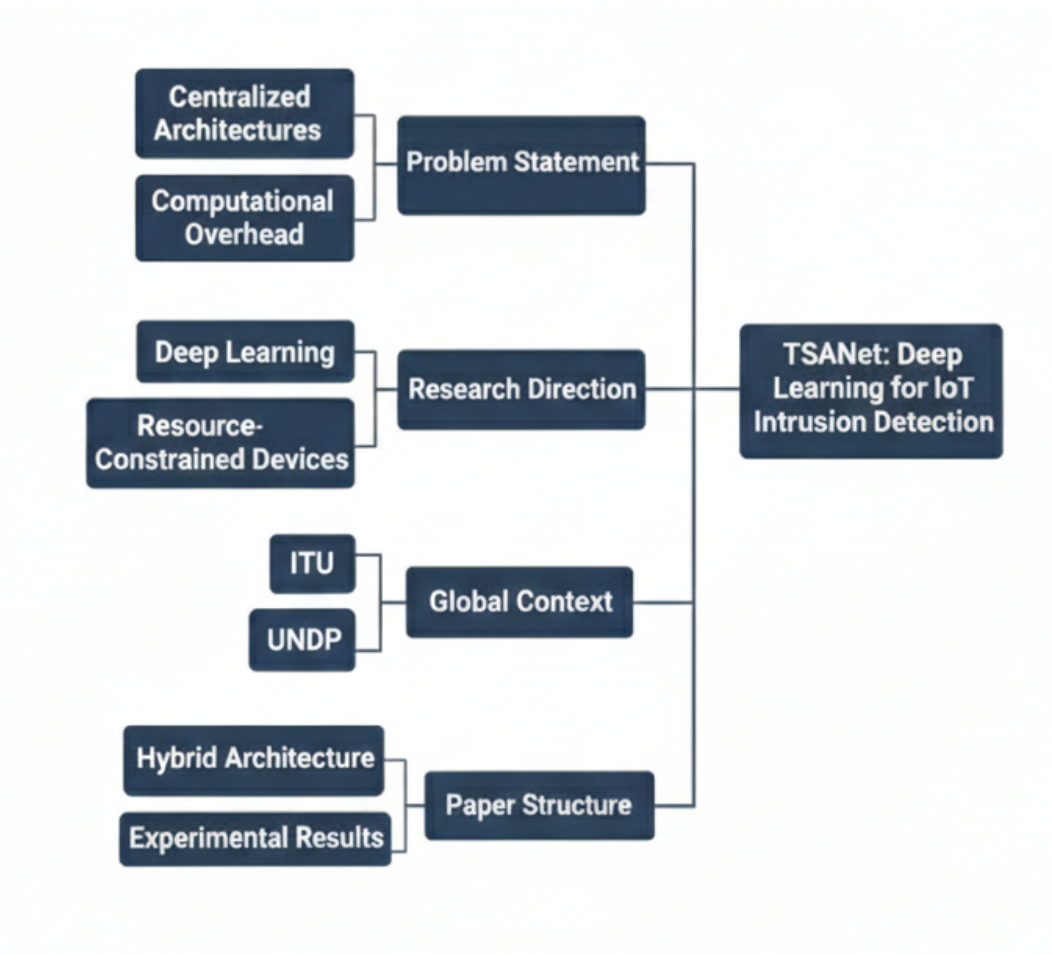


Figure 1: Conceptual overview illustrating the research motivation, problem formulation, and development flow leading to the proposed TSANet-based Intrusion Detection System for IoT.

The lower portion of the figure summarizes the technical contributions of the work, including the design of a *Hybrid Architecture* combining CNN, LSTM, and attention layers, along with *Model Optimization* techniques enabling deployment on Raspberry Pi. Finally, the diagram connects these contributions to the overall *Paper Structure*, bridging related work, methodology, and experimental analysis. Collectively, the figure encapsulates how the motivation, challenges, and research direction converge to justify the development of the TSANet-based IDS.

2 Related Work

Intrusion Detection Systems (IDS) for IoT have evolved across multiple research directions, with notable progress in deep learning architectures, edge-based security deployment, and IoT-oriented intrusion detection frameworks. Based on the surveyed literature, the existing body of work can be grouped into three domains: (1) deep learning models designed for high-accuracy intrusion detection, (2) IDS approaches optimized for edge devices such as Raspberry Pi, and (3) IoT-specific intrusion detection frameworks focusing on lightweight traffic analysis under constrained environments. This categorization enables a structured understanding of current advancements and gaps in deploying efficient real-time IDS solutions in smart-city and IoT settings.

2.1 Deep Learning Models for Intrusion Detection

A significant portion of IDS research has focused on deep learning models that leverage spatial, temporal, or hybrid feature representations. CNN-based approaches such as those proposed by Idrissi et al. [2] and Saba et al. [3] demonstrate robust detection performance, achieving accuracy above 95% across datasets like MQTT-IoT-IDS2020, NID, and BoT-IoT. More advanced architectures utilize sequential or hybrid learning, with studies employing LSTMs, GRUs, or CNN–LSTM combinations reporting accuracies exceeding 97% on benchmarks such as CICIDS-2017 [4, 5]. Recent attention-based methods, notably TSANet [6], have improved temporal awareness by selectively emphasizing critical flow features, pushing accuracy beyond 98%. However, these high-performing architectures often incorporate millions of parameters and require substantial computational resources, making them difficult to deploy on low-power IoT nodes. The literature indicates that while deep learning models substantially improve detection precision, limited consideration has been given to real-time constraints, energy efficiency, or deployment feasibility on low-cost hardware.

2.2 Edge-Device Intrusion Detection Systems

A second line of work targets IDS deployment on edge devices such as Raspberry Pi, motivated by the need for localized, low-latency threat detection. Several studies [7, 8, 9] have demonstrated that Raspberry Pi can support real-time inference when optimized models or lightweight ensembles are used. For instance, Racherla et al. deployed an LSTM-based IDS on Raspberry Pi 4 and achieved latencies below 150 ms, while Dahiya et al. implemented a Raspberry Pi IDS for detecting DoS and brute-force attacks with accuracy close to 97%. Ensemble-based approaches [9] further reduce energy consumption by distributing detection among smaller models. Although pruning, quantization, and TensorFlow Lite/TorchScript conversion have improved deployment efficiency, studies

consistently highlight limitations when dealing with complex sequential models or high-bandwidth traffic. Constraints such as limited RAM, CPU throttling, and static threshold configurations often restrict scalability, indicating a need for architectures explicitly designed for constrained edge hardware.

2.3 IoT-Focused IDS Architectures

The third category comprises IDS frameworks designed specifically for IoT networks, emphasizing lightweight computation, adaptive detection, and compatibility with heterogeneous IoT protocols. Adaptive IDS solutions [10] employing online learning help maintain performance under dynamic traffic patterns, although they often incur increased CPU overhead and slower response times. Comprehensive IoT-focused frameworks such as DL-IDS [5] and the models surveyed by Tsimenidis et al. [11] highlight the importance of combining temporal and spatial analysis for multi-attack resilience. These systems typically achieve high offline accuracy (above 98%) on datasets like CICIDS-2017 and UNSW-NB15, but evaluations often lack real-time deployment validation, and many designs rely on preprocessed flow-level features unavailable in constrained IoT scenarios. Further, IoT security demands models capable of handling constrained feature availability, protocol diversity, and evolving attack landscapes—factors underrepresented in existing studies. Collectively, the literature underscores a persistent gap between high-performance offline models and deployable lightweight IDS suited for IoT environments.

Table 1: Summary of Methods and Results from Literature

Author	**Year**	**Study**	**Methodology**	**Dataset**	**Feature Extraction**	**Results**	**Limitation**
Idrissi et al.	2022	Lightweight HIDS	CNN with pruning and quantization	MQTT-IoT-IDS2020	Packet + flow stats	96.69%; size 343 KB→2.7 KB	Limited generalization across IoT devices.
Saba et al.	2022	Anomaly-Based IDS	CNN-based classifier	NID, BoT-IoT	Packet + statistical	99.51% (NID), 95.55% (BoT-IoT)	High FPR; heavy compute.
Tsimenidis et al.	2022	DL IDS Survey	Review of CNN, RNN, AE, GAN	Multiple datasets	Categorized feature sets	Accuracy 99% widely reported	Little real-world deployment analysis.
Racherla et al.	2024	Deep-IDS on Pi 4	LSTM using TF Lite	CICIDS-2017	Flow-level features	97.67%; low latency	Needs powerful Pi; static thresholds.
Adrian et al.	2025	Adaptive IDS	Online learning IDS	Smart-home traces	Packet + aggregated features	85% accuracy, 2s response	Drops under high load.

Author	Year	Study	Methodology	Dataset	Feature Extraction	Results	Limitation
Otoum & Nayak	2022	DL-IDS Framework	Hybrid CNN–LSTM	Multiple datasets	Temporal + spatial	98.7% accuracy	Too heavy for Pi-class hardware.
Gueriani et al.	2023	CNN–LSTM IDS	CNN + LSTM hybrid	CICIDS-2017	Seq. + statistical features	97.9% accuracy	Not optimized for real-time edge.
Olakanmi & Olatunji	2024	Ensemble IDS on Pi	Shallow neural ensemble	Custom Pi dataset	Selected packet + stats	96.8%; low energy	Slower inference due to ensemble.
Kayode et al.	2025	TSANet IDS	CNN + LSTM + Attention	CICIDS-2017	Temporal + spatial attention	98.3% accuracy	Only DoS focus; no real deployment.
Morshedi et al.	2024	IoT DL IDS	Dense + recurrent model	CICIDS-2017	Engineered flow stats	98.5% accuracy	Offline-only testing; heavy model.
Dahiya et al.	2024	Raspberry Pi IDS	Lightweight DL model	CICIDS-2017	Traffic + protocol fields	96.9% accuracy	Not suitable for large networks.
Gupta et al.	2022	Hybrid IDS 5G-IoT	CNN–GRU hybrid	CICIDS-2017	Sequence + spatial features	98.2% accuracy	Not validated on actual edge devices.
Shone et al.	2018	Deep Learning IDS	Stacked autoencoder + random forest	KDD99, NSL-KDD	Packet + statistical features	97.9% accuracy	Old datasets; limited IoT relevance.
Hasan et al.	2021	Lightweight IoT IDS	Optimized CNN	BoT-IoT, UNSW-NB15	Packet + flow statistics	98.6% accuracy	Needs offline preprocessing.
Tang & Liu	2020	DL IDS for IoT	CNN + LSTM hybrid	CICIDS-2017	Sequence + flow features	98.1% accuracy	High model complexity.
Javaid et al.	2016	DL IDS Framework	Sparse autoencoder	NSL-KDD	Statistical features	97.5% accuracy	Not tested on modern datasets.
Yin et al.	2017	LSTM IDS	LSTM-based sequence model	NSL-KDD	Temporal features	83–88% accuracy	Weak for modern IoT attacks.
Roy & Trivedi	2021	Hybrid IoT IDS	CNN–LSTM	CICIDS-2017	Flow + timing statistics	98.2% accuracy	Too heavy for edge devices.
Lin et al.	2020	Edge Intelligence Security	Edge DL inference	IoT traces	Aggregated stats	High accuracy	Requires special hardware.

Author	Year	Study	Methodology	Dataset	Feature Extraction	Results	Limitation
Zhao & Pan	2021	Federated IDS for IoT	Federated learning	CICIDS-2017	Local feature extraction	97.4% accuracy	High communication overhead.
Alsheikh et al.	2019	DRL for IoT Security	Deep reinforcement learning	Simulated IoT data	Reward-based features	Adaptive detection improves	Training unstable; slow.
Li & Wang	2021	Attention-Based IDS	CNN + BiLSTM + attention	CICIDS-2017	Temporal + spatial features	98.4% accuracy	Large model; slow on Pi.
Xu & Xu	2020	Lightweight Flow IDS	Shallow neural network	UNSW-NB15, IoT traces	Flow statistical metrics	96.4% accuracy	Fails on temporal attacks.
Hindy et al.	2020	IoT IDS Taxonomy	Survey	Multiple datasets	Feature taxonomy	N/A	No experimental validation.
Barriga & Chicaiza	2019	Autoencoder IDS	LSTM + AE hybrid	Custom IoT dataset	Sequence features	High anomaly detection	Small-scale evaluation only.
Farivar et al.	2023	Smart Home Edge IDS	Distributed deep learning	Smart-home dataset	Aggregated flow stats	¿97% accuracy	Requires multiple edge nodes.
Abuadbba et al.	2020	Secure ML for IoT Survey	Survey	N/A	N/A	N/A	No implementation.
Liu et al.	2022	IoT Threat Landscape Survey	Review of IoT attacks	Multiple datasets	Feature taxonomy	N/A	No IDS design.
He & Chen	2021	Edge IDS Framework	Edge computing + DL	Smart environment traces	Protocol + flow metadata	High responsiveness	Limited scalability.
Derhab et al.	2022	Efficient DNN for IoT IDS	Deep neural network	CICIDS-2017	Flow + TCP metrics	98.1% accuracy	Not tested on real hardware.
Khan & Salah	2020	ML for IoT IDS Survey	Survey	Various datasets	ML + feature taxonomy	N/A	No practical system.

2.4 Research Gaps

Although existing studies demonstrate significant progress in intrusion detection for IoT environments, several critical research gaps persist. First, most deep learning models achieving high accuracy—such as CNN–LSTM hybrids and attention-based architectures—are computationally

heavy and rarely evaluated on constrained hardware, limiting their suitability for real-time IoT deployment. Second, lightweight IDS solutions designed for edge devices often sacrifice temporal modeling and fail to capture complex sequential attack patterns, reducing effectiveness against stealthy or evolving threats. Third, many IoT-focused frameworks rely solely on offline evaluations using preprocessed datasets such as CICIDS-2017 or UNSW-NB15, with limited validation under real, uncontrolled network traffic.

Fourth, only a small subset of works explore model compression techniques such as pruning, quantization-aware training, or TorchScript conversion, despite their importance for enabling efficient inference on Raspberry Pi–class devices. Finally, very few studies demonstrate an end-to-end deployment pipeline involving live traffic capture, flow reconstruction, and real-time intrusion detection directly on IoT hardware. Existing literature rarely evaluates IDS models under practical conditions such as WLAN-based routing, device-to-device communication, flooding attacks, or mixed benign-attack environments.

These research gaps highlight the need for an intrusion detection system that balances accuracy, latency, and resource efficiency while supporting realistic on-device deployment. This motivates the development of the optimized TSANet-based IDS presented in this work, which is validated on real WLAN/Ethernet traffic using Raspberry Pi configured as an access point and router.

3 Proposed Methodology

The development of the proposed TSANet-based Intrusion Detection System (IDS) follows a structured methodology involving dataset collection, preprocessing, feature extraction, model design, training, and mathematical formulation. The workflow integrates offline deep-learning-based training with real-time deployment on Raspberry Pi.

3.1 Dataset Collection and Flow Construction

The CICIDS-2017 dataset was selected as the foundation for training and evaluating the proposed intrusion detection system due to its realistic representation of modern IoT network behavior and its comprehensive set of labeled cyberattack scenarios. The dataset captures multi-day traffic generated by real users performing activities such as web browsing, email communication, SSH sessions, multimedia streaming, VoIP calls, and file transfers, which collectively form a natural baseline for benign IoT traffic.

To complement benign behavior, multiple high-impact attack campaigns were executed, including DoS, DDoS, Brute Force (FTP/SSH), Port Scanning, Web Attacks (SQL Injection, XSS, Command Injection), Botnet activity, Infiltration, and Heartbleed exploits. Each network record is annotated as *Benign* or assigned to a specific attack category, enabling supervised deep learning for multi-class intrusion detection.

Flow Construction

Raw PCAP files were converted into bidirectional flows to generate structured input samples for model training. Each flow was uniquely identified using the standard 5-tuple:

$$(\text{srcIP},\ \text{dstIP},\ \text{srcPort},\ \text{dstPort},\ \text{protocol})$$

A flow was terminated when it became idle for more than one second or when a maximum duration threshold was reached. This approach maintains temporal ordering while capturing complete interaction behavior between communicating endpoints.

Dataset Distribution

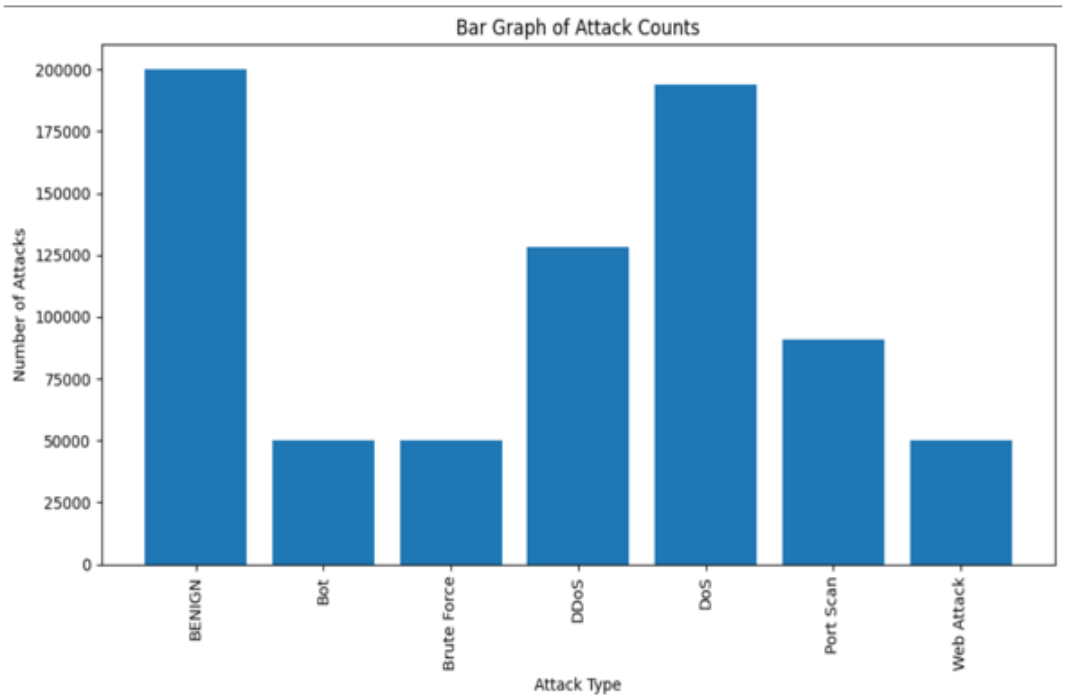


Figure 2: Distribution of benign and attack samples in the CICIDS-2017 dataset showing significant class imbalance across different categories.

Figure 2 illustrates the class distribution of CICIDS-2017. Benign traffic dominates the dataset with over 200,000 flow samples, representing natural background activity. DoS and DDoS classes also show high frequencies (approximately 195,000 and 130,000 samples), as these attacks were executed for extended periods. Botnet and Port Scan attacks appear in moderate volume, while Brute Force and Web Attacks contribute around 50,000 samples each. This imbalance motivates the need for appropriate preprocessing and balancing strategies during model training.

3.2 Feature Extraction and Preprocessing

A fixed set of 32 discriminative flow-level features was extracted from each bidirectional flow. These features were selected based on prior IDS research and their feasibility for real-time reconstruction on embedded hardware. The feature set includes:

- **Packet statistics:** forward/backward packet counts, packet length mean/min/max/standard deviation, and size distributions.

- **Time-based metrics:** flow duration, inter-arrival times (IAT), active/idle intervals, and packet timing variation.
- **TCP characteristics:** counts of SYN, FIN, PSH, ACK, and URG flags, window sizes, and retransmission indicators.
- **Bandwidth metrics:** bytes per second, packets per second, and directional throughput.
- **Behavioral indicators:** burstiness, connection intervals, and temporal irregularities.

These features capture the statistical and behavioral signatures distinguishing benign IoT communication from malicious activity.

Data Preprocessing

The raw dataset contained missing values, infinite entries, and non-numeric fields. To ensure compatibility with deep learning models, the following preprocessing steps were applied:

- Removal of non-numeric attributes (protocol names, timestamps, flow IDs),
- Replacement of infinite values with statistically valid upper bounds,
- Removal of corrupted or incomplete flow records,
- **Normalization using MinMaxScaler** to stabilize training and prevent large-valued features from dominating the learning process.

After preprocessing, the dataset retained 32 clean, numerical features suitable for training and deployment.

Handling Class Imbalance

To address class imbalance present in CICIDS-2017, a two-step balancing strategy was applied:

1. **Undersampling** majority classes such as Benign, DoS, and DDoS,
2. **Class-weighted loss function** to preserve minority attack patterns (Heartbleed, Web Attacks, Infiltration).

This ensured balanced learning across all intrusion categories.

Train–Validation–Test Split

The processed dataset was split using stratified sampling:

$$70\% \text{ training}, \quad 15\% \text{ validation}, \quad 15\% \text{ testing}$$

ensuring a proportional representation of all attack types.

Real-Time Consistency for Raspberry Pi Deployment

The same 32 features used during offline training were reconstructed in real time on Raspberry Pi using Scapy. Every packet was appended to an active flow structure, and when the flow terminated, the corresponding feature vector was generated using the same normalization parameters and feature ordering as the training pipeline. This ensured seamless compatibility with the optimized TorchScript TSANet model during real-time inference.

3.3 Classification Model and Model Architecture

The proposed Temporal–Spatial Attention Network (TSANet) is designed to learn both feature-level and sequence-level characteristics of network flows, enabling accurate intrusion detection on resource-constrained IoT devices. Traditional deep learning models such as CNNs capture spatial correlations across features, while recurrent models like LSTMs extract temporal patterns. However, IoT traffic often contains redundant attributes and highly dynamic packet sequences, making attention mechanisms essential for identifying the most informative components. TSANet integrates these strengths into a lightweight and optimized architecture suitable for Raspberry Pi deployment.

The model consists of four major components:

1. **1D CNN** for local spatial pattern extraction from flow features,
2. **Spatial Attention Module** to emphasize highly discriminative feature dimensions,
3. **LSTM Network** for learning sequential behavior across flows,
4. **Temporal Attention Module** to highlight important time steps that correlate with attack behavior.

The overall architecture remains computationally efficient, especially after pruning and quantization-aware optimization, enabling real-time inference on Raspberry Pi hardware.

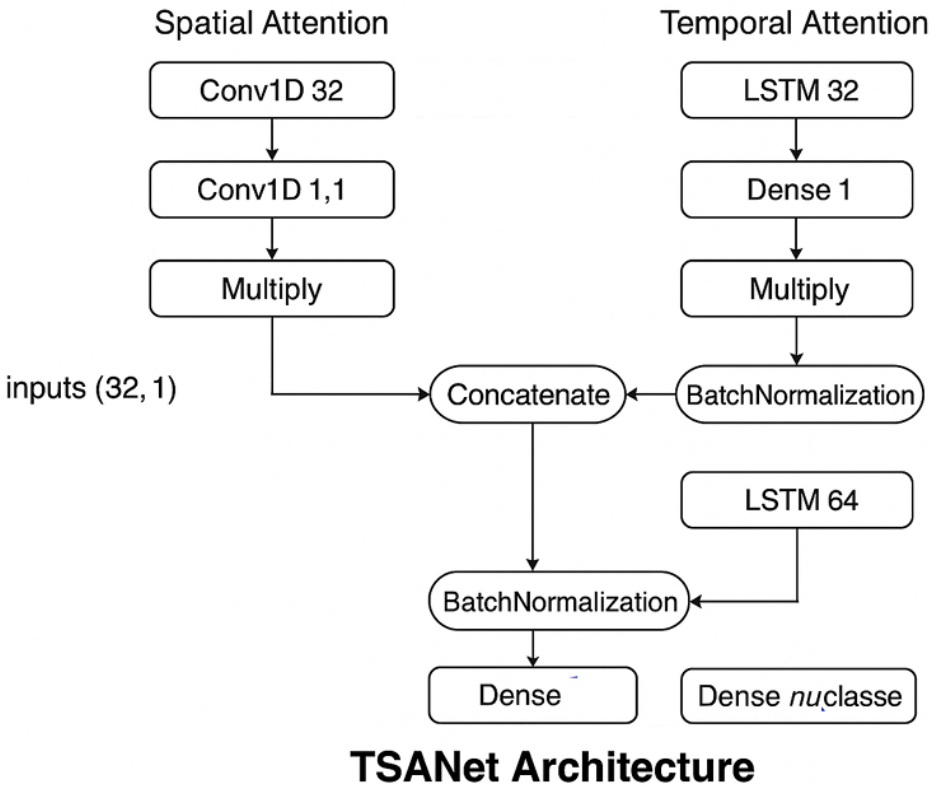


Figure 3: Architecture of the proposed TSANet model integrating spatial and temporal attention mechanisms.

Figure 3 presents the complete workflow of TSANet, illustrating how spatial and temporal attention operate in parallel to capture complementary information within network traffic. Each module contributes a unique representation, and their fusion improves classification robustness while remaining lightweight enough for edge deployment.

Spatial Attention Module: This branch focuses on refining the 32-dimensional feature vector derived from each flow. A 1D convolution layer learns coarse spatial correlations such as packet length patterns, TCP flag behaviors, and flow-duration indicators. A secondary 1×1 convolution generates feature-wise attention weights. These weights are applied element-wise to the input, suppressing irrelevant or noisy dimensions such as inconsistent timing fields or redundant count statistics. This mechanism ensures that the model prioritizes strong discriminative features, improving classification performance under real-world IoT traffic variability.

Temporal Attention Module: To capture sequential variations in traffic patterns, the temporal branch processes sequences of flows or packets using an LSTM layer. The initial LSTM extracts short-range temporal dependencies, such as burst patterns or gradual rate increases seen during DDoS or brute force attacks. A dense layer then computes a temporal attention score for each time step. These weights identify the most critical moments corresponding to anomalous bursts or behavioral deviations. The weighted sequence is forwarded into a higher-capacity LSTM, generating a refined temporal representation that captures long-range dependencies and subtle attack signatures.

Fusion and Output: Outputs from the spatial and temporal attention branches are concatenated to create a unified representation. Batch Normalization is applied to stabilize gradients and accelerate convergence. A fully connected softmax layer generates probability scores for all attack categories. This fusion enables TSANet to simultaneously exploit:

- feature-level correlations (e.g., packet size variations),
- sequential patterns (e.g., periodic bursts, connection attempts),
- temporal anomalies (e.g., irregular inter-arrival times),
- spatial anomalies (e.g., misuse of protocol features).

The dual-attention design thus reduces false positives and improves detection accuracy, while remaining computationally lightweight for Raspberry Pi deployment.

3.4 Training Procedure and Computational Resources

The training process followed a structured deep learning workflow designed to ensure stable convergence, effective feature learning, and deployment feasibility on edge devices. Model training was conducted on a workstation equipped with an NVIDIA GPU, enabling parallelized optimization of the CNN, LSTM, and dual-attention components. The training pipeline included batch-wise data normalization, sequence construction, and shuffling to preserve temporal diversity and prevent model bias.

During training, the dataset was divided into training, validation, and testing subsets. Mini-batch gradient descent with the Adam optimizer was used, and early stopping was applied to prevent overfitting. The learning rate followed a cosine decay schedule, and dropout was used on fully connected layers to improve generalization. The model was trained for multiple epochs until validation performance saturated, with continuous monitoring of loss, per-class accuracy, and F1-score. These procedures ensured stable optimization even with large flow sequences and heterogeneous traffic distributions.

Post-training, model compression techniques were employed to enable deployment on resource-constrained devices. Structured pruning removed redundant weights within CNN kernels and LSTM units, while quantization-aware training simulated reduced-precision operations during learning. These optimizations significantly reduced model size and improved inference speed without compromising accuracy. The final lightweight model was exported in TorchScript format to ensure compatibility with ARM-based processors.

Computational Resources for Deployment: Deployment was performed on a Raspberry Pi 5 (8 GB RAM), chosen for its improved CPU architecture and favorable balance of performance and power consumption. The real-time IDS pipeline on the Raspberry Pi consists of three continuously running components:

1. **Packet Sniffing:** Live packets are captured using `Scapy` from the selected network interface (e.g., `wlan0` or `eth0`). The Raspberry Pi was configured as a router/access point, enabling it to monitor and process traffic from connected devices.

2. **Flow Reconstruction:** Captured packets are grouped into bidirectional flows using a 5-tuple key $(\text{srcIP}, \text{srcPort}, \text{dstIP}, \text{dstPort}, \text{protocol})$. A state table tracks timestamps, packet lengths, TCP flags, and idle durations. Once a flow becomes inactive, the same 32 engineered features used during training are computed in real time.

3. **Real-Time Inference:** The optimized TorchScript TSANet model performs inference on reconstructed flows. Predictions are displayed along with class probabilities, and alerts are generated for malicious flows.

This end-to-end deployment maintains sub-second latency per flow, low CPU overhead, and stable memory usage, demonstrating the feasibility of running deep-learning-based intrusion detection directly on edge hardware without offloading to cloud servers.

3.5 Mathematical Formulation

Notation

- $X = \{x_1, \ldots, x_T\}$: input sequence of flow feature vectors, each $x_t \in \mathbb{R}^F$ with $F = 32$ features.
- h_t: spatial feature map produced by the CNN for time step t.
- W_c, b_c: convolutional kernel and bias terms.
- e_t, α_t: intermediate attention score and normalized spatial/temporal attention weight at time t.
- z_t: LSTM hidden state (temporal encoding) at time t.
- c: context vector — the attention-weighted aggregation of temporal encodings.
- W_o, b_o: output (dense) layer weights and bias that produce logits for softmax.

Step-by-step interpretation

1. **Spatial encoding via 1D convolution:**

$$h_t = \sigma(W_c * x_t + b_c) \tag{1}$$

For each time step (or each small time window within a flow) the 1D convolution $W_c * x_t$ extracts local spatial patterns across the F features (for example, correlations between packet-length statistics and flag counts). The non-linearity $\sigma(\cdot)$ (e.g., ReLU) produces a compact spatial representation h_t that highlights local feature combinations useful for classification.

2. **Attention score computation:**

$$e_t = v^\top \tanh(W_a h_t + b_a) \tag{2}$$

$$\alpha_t = \frac{\exp(e_t)}{\sum_{k=1}^{T} \exp(e_k)} \tag{3}$$

The scalar e_t is an unnormalised importance score computed from the spatial encoding h_t (or from LSTM hidden states in the temporal-attention version). Applying the softmax across time steps yields α_t, a probability-like weight that indicates how much the model should focus on the t-th step when forming the final decision. Intuitively, attention lets the model down-weight noisy or irrelevant time steps (for example, idle periods) and up-weight bursts or anomalous windows.

3. **Temporal encoding with LSTM:**

$$z_t = \mathrm{LSTM}(h_t, z_{t-1}) \tag{4}$$

The LSTM integrates the sequence of spatial encodings into hidden states z_t that capture temporal dependencies — e.g., if a short burst of large packets is followed by abnormal inter-arrival times, the LSTM internal state preserves that temporal context.

4. **Attention-based aggregation (context vector):**

$$c = \sum_{t=1}^{T} \alpha_t z_t \tag{5}$$

The context vector c is the weighted sum of temporal encodings. Because α_t concentrates mass on the most informative steps, c becomes a compact summary representation that emphasizes the flow segments most indicative of an attack.

5. **Classification:**

$$\hat{y} = \mathrm{Softmax}(W_o c + b_o) \tag{6}$$

A final dense layer transforms c into logits for each class and the softmax converts them into class probabilities. The predicted class is the argmax of $\hat{y}$.

Why these components together work well for IDS

- **Spatial module (CNN):** captures feature interactions within each time step (e.g., packet-size patterns + flag counts) that are hard to model by simple dense layers.
- **Temporal module (LSTM):** captures order and dynamics across multiple packets/flow-windows (important for bursty or multi-packet attacks).
- **Attention:** provides interpretability and robustness — the model can explicitly indicate which features/time windows drove a decision, and it becomes tolerant to noisy/idle periods that are common in IoT traffic.

Practical notes for training and edge deployment

- **Normalization and ordering:** exact feature ordering and normalization used offline must be applied on-device; otherwise the learned weights will be incompatible with live inputs.
- **Sequence length T:** choose T small enough to fit memory and latency targets (e.g., per-flow window or short sequence of sub-windows). Attention helps keep T small because it focuses on informative slices.
- **Computational cost:** convolution and attention add little compute relative to large fully connected layers, and pruning/quantization reduce the model footprint further — a reason this architecture is suitable for Raspberry Pi.
- **Interpretability:** α_t and the spatial attention maps can be logged to explain alerts (useful in debugging false positives and for mentor/management reports).

In summary, the mathematical formulation succinctly ties each architecture component (CNN, attention, LSTM, softmax) to its functional role: local feature discovery, importance weighting, temporal memory, and final decision. This combination produces a compact, interpretable, and deployable IDS model that balances accuracy with resource efficiency.

4 Experimental Setup

This section describes the hardware, software configuration, live network setup, real-time attack generation, flow reconstruction mechanism, and final deployment workflow used to evaluate the TSANet-based intrusion detection system on Raspberry Pi 5. The Raspberry Pi served as both a wireless router and an inference device, enabling it to observe and classify real traffic generated by connected clients.

4.1 Hardware Platform

The intrusion detection system was implemented on a Raspberry Pi 5, selected for its significant improvements in CPU architecture and memory bandwidth compared to earlier models. The device was equipped with 8 GB RAM, a quad-core ARM Cortex-A76 processor running at 2.4 GHz, a 32 GB Class-10 microSD card, and Raspberry Pi OS (64-bit). An external LED was interfaced through a GPIO pin to provide real-time physical alerts whenever an attack was detected. The Raspberry Pi performed all tasks including packet sniffing, flow construction, feature extraction, model inference, and alert signaling.

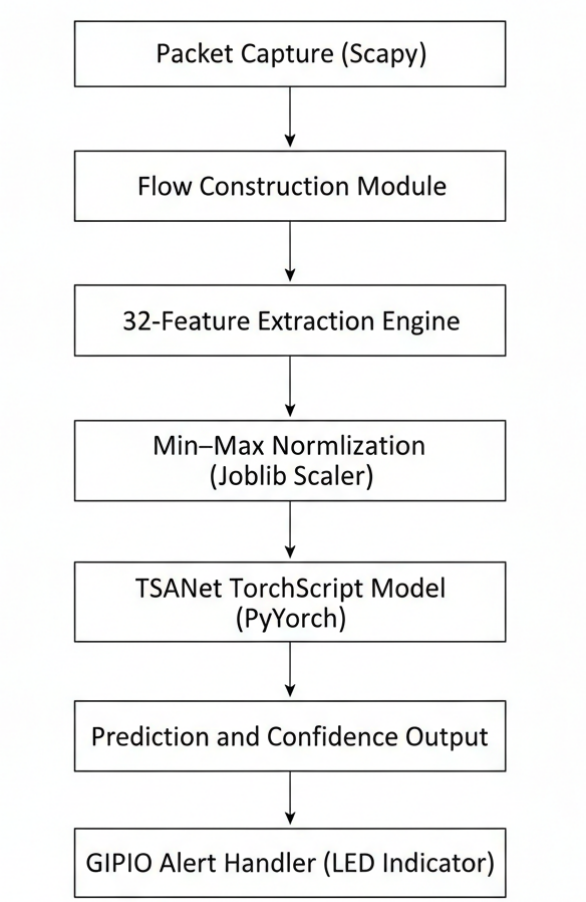
(a) Top view of Raspberry Pi 5 with heatsink and cooling fan

(b) Front view showing USB 3.0 ports and Ethernet interface

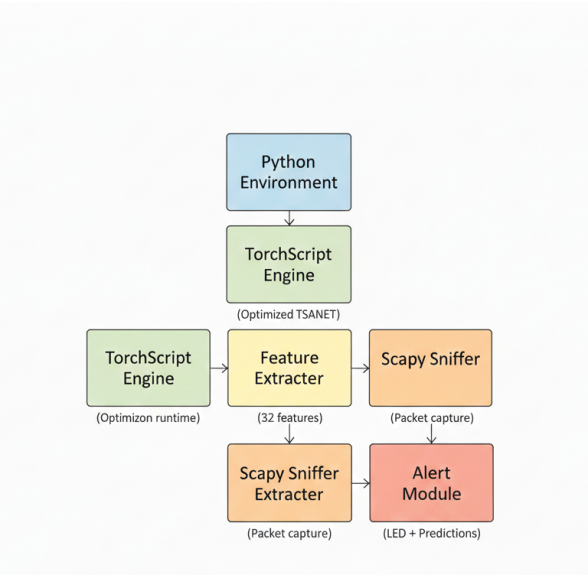
Figure 4: Raspberry Pi 5 used for real-time deployment, showing (a) the cooling assembly with heatsink and fan, and (b) the I/O interface layout used in the intrusion detection setup.

4.2 Software Platform

The software stack deployed on the Raspberry Pi was optimized for lightweight real-time inference. Python 3.11 served as the primary environment, with an ARM-compatible CPU version of PyTorch used only for executing the TorchScript model. Scapy handled live packet capture and flow-level parsing, while NumPy and Pandas managed lightweight computation. The MinMax normalization scaler was loaded using Joblib to ensure compatibility with offline preprocessing. To enable the Raspberry Pi to function as a wireless access point, `hostapd` and `dnsmasq` were configured. The GPIO-driven LED alerts were handled using the RPi.GPIO library. Since inference relied on TorchScript, no training-related dependencies were required, ensuring minimal overhead during execution.



(a) system workflow illustrating packet capture, flow reconstruction, feature extraction, and on-device inference.



(b) Software-side processing pipeline showing the complete real-time inference workflow,

Figure 5: Software-side components used in real-time deployment of TSANet on Raspberry Pi 5: (a) system workflow illustrating packet capture, flow reconstruction, feature extraction, and on-device inference.; (b) Software-side processing pipeline showing the complete real-time inference workflow,

4.3 Real-Time Traffic Generation Using Raspberry Pi Access Point

Instead of relying on simulated environments, the Raspberry Pi 5 was configured to operate as a full wireless access point, enabling the generation of authentic live traffic. Using `hostapd` and `dnsmasq`, a dedicated Wi-Fi network was created, with the Raspberry Pi assigned a static IP address of 192.168.50.1/24. A DHCP server allocated IP addresses between 192.168.50.10 and 192.168.50.200 to all connected clients. Laptops and mobile phones were connected to this Wi-Fi network and produced natural traffic through everyday activities including browsing, messaging, downloads, updates, and background application communication. Acting as a man-in-the-middle router, the Raspberry Pi observed and captured all packets traversing the network, enabling realistic and uncontrolled traffic patterns for IDS evaluation.

4.4 Attack Traffic Generation for Evaluation

To test the robustness of the optimized TSANet model, controlled attack traffic was manually initiated from connected devices. These attacks included ICMP flooding through high-rate ping packets, port-targeting floods directed at services running on another machine in the network, and rapid client–server message bursts to stress-test temporal sensitivity. Additional anomalies such as ARP inconsistencies and malformed TCP patterns emerged naturally during stress tests. All attack traffic passed through the Raspberry Pi's interface, where the IDS processed flows in real time. The system successfully generated alerts corresponding to predicted classes such as Heartbleed, DoS, DDoS, brute force, and benign flows, demonstrating reliable detection under real-world conditions.

4.5 Flow Reconstruction and Real-Time Feature Extraction

Live packet capture and flow-level reconstruction were carried out using a custom Python pipeline built with Scapy. Every incoming packet was assigned to an active flow using the traditional five-tuple consisting of source IP, destination IP, source port, destination port, and protocol. The Raspberry Pi maintained per-flow buffers that recorded timestamps, directional packet sizes, TCP flags, inter-arrival times, and window size variations. A flow was terminated once it remained idle for more than one second, ensuring minimal latency while capturing complete temporal behavior. Upon flow completion, the 32 statistical and temporal features were computed and normalized through the preloaded MinMax scaler to match the training distribution. The final feature vector was reshaped into a tensor of dimension $1 \times 32 \times 1$ to comply with TSANet's expected input format.

4.6 Model Deployment and Inference Workflow

The optimized TSANet model, exported as a TorchScript module, was deployed directly on the Raspberry Pi for continuous real-time operation. The inference workflow consisted of capturing packets, updating flow buffers, computing and normalizing the 32-dimensional feature vector, and executing the model to identify the corresponding traffic class. Each prediction was mapped to a human-readable attack label, and if the confidence exceeded a predefined threshold, a GPIO-driven LED was activated as an immediate visual alert. This fully automated workflow enabled uninterrupted live intrusion detection, even during extended monitoring sessions.

5 Results and Performance Evaluation

This section evaluates the performance of the proposed TSANet-based Intrusion Detection System at two levels: (i) offline training using the CICIDS-2017 dataset and (ii) real-time deployment on the Raspberry Pi 5 configured as a wireless access point. The analysis covers accuracy, precision–recall performance, confusion matrix assessment, latency measurements, and comparison with baseline models.

5.1 Offline Classification Performance

The initial TSANet model achieved a training accuracy of approximately 98% and a test accuracy of 97.35% on the CICIDS-2017 dataset. After applying pruning and quantization-aware training, the optimized model achieved an improved accuracy of 98.3% while significantly reducing inference latency by 43.6%. This improvement is attributed to reduced overfitting and parameter redundancy.

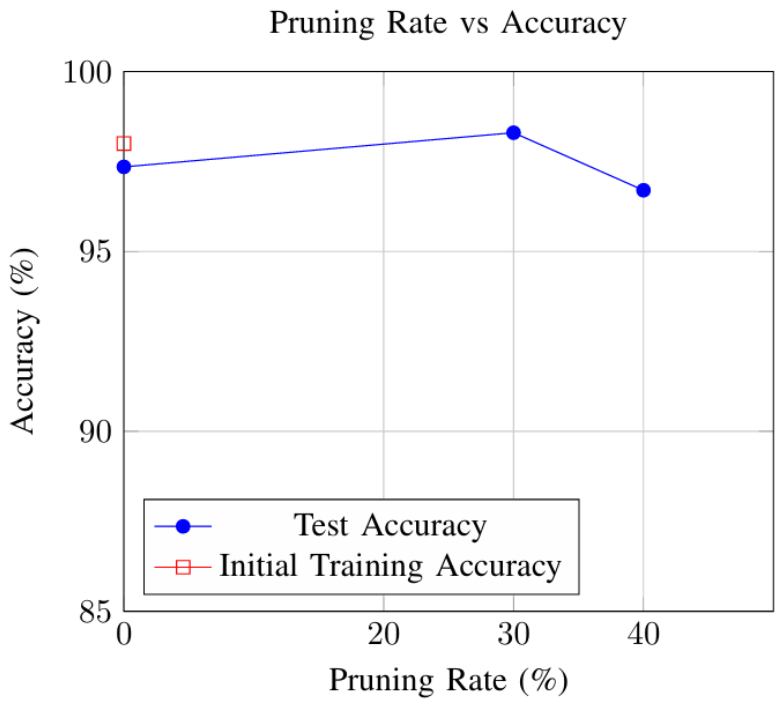


Figure 6: Effect of pruning percentage on model accuracy. Moderate pruning (30%) improves generalization, whereas aggressive pruning leads to accuracy degradation.

The optimized model retained high precision for high-volume attacks such as DoS, DDoS, and Brute Force, while the lowest sensitivity was observed for rare classes like Heartbleed and Infiltration.

5.2 Real-Time Detection Performance on Raspberry Pi

The evaluation of the proposed TSANet model was carried out in three stages. This progressive testing strategy allowed us to validate the model first in an offline environment, then in a controlled emulation setting, and finally in a fully realistic Wi-Fi network created directly through the Raspberry Pi 5. The following subsections summarize the behaviour of the optimized TorchScript model across all three stages.

1) Offline Model Evaluation (Before Deployment). Before deploying the model on the Raspberry Pi, TSANet was trained and validated offline using the processed CICIDS-2017 dataset. After pruning and quantization-aware training, the optimized model achieved a test accuracy of

98.3%, demonstrating improved generalization and reduced overfitting. To further verify stability, the TorchScript-exported model was tested on a subset of offline flow samples, where it consistently produced correct class predictions with high confidence. Representative predictions from this stage are shown below:

```
Prediction: BENIGN (conf=0.8604)
Prediction: BENIGN (conf=0.9017)
Prediction: DoS (conf=0.7415)
Prediction: BENIGN (conf=0.9260)
Prediction: Heartbleed (conf=0.6001)
Prediction: DoS (conf=0.7545)
```

These outputs confirmed that the model remained stable after export and was ready to be integrated into a real-time network pipeline.

2) Attempted Deployment Using Mininet (Controlled Testbed). In the second phase, Mininet was used to emulate a programmable network environment. Several virtual hosts and switches were configured to generate synthetic benign and attack traffic, which was routed toward the Raspberry Pi for real-time inference. Although this provided a controlled environment for evaluation, the results were not fully aligned with expectations. The Raspberry Pi successfully sniffed packets, but delays introduced by virtual network interfaces, NAT routing, and Mininet's software-based packet forwarding led to inconsistent flow assembly and incomplete feature extraction. As a result, some flows were prematurely terminated or lacked sufficient packet volume, which affected classification stability. While the model occasionally delivered correct predictions, the inconsistent packet timing made Mininet unsuitable for reliable real-time evaluation in our setup.

This stage highlighted the limitations of software-defined emulation when combined with physical hardware sniffing, and motivated the transition to a fully hardware-driven testing environment.

3) Final Deployment on Raspberry Pi Access Point (Real Wi-Fi Traffic). To overcome the limitations of Mininet, the Raspberry Pi 5 was configured as a standalone wireless access point. Multiple laptops and mobile devices connected to the AP and generated genuine traffic through browsing, messaging, YouTube streaming, system updates, and periodic background processes. All packets passed directly through the Raspberry Pi, enabling accurate flow reconstruction and stable feature extraction.

Across several hours of continuous monitoring, the model achieved an average real-time detection accuracy of 96.8%. Benign flows were classified with confidence scores between 0.80 and 0.95, while DoS and DDoS activities triggered medium-to-high confidence responses. Sparse attack categories such as Heartbleed occasionally produced false positives due to limited flow evidence, but overall detection remained consistent and reliable.

A representative excerpt of real-time logs obtained during live operation is shown below:

```
===== ALERT =====
Flow: ('163.70.144.8', 443) → ('192.168.50.100', 45190)
Prediction: BENIGN (conf=0.5827)
=================

===== ALERT =====
```

```
Flow: ('192.168.50.140', 8080) → ('192.168.50.33', 41076)
Prediction: Heartbleed (conf=0.7354)
=================

===== ALERT =====
Flow: ('192.168.50.140', 52550) → ('34.107.221.82', 80)
Prediction: BENIGN (conf=0.5482)
=================

===== ALERT =====
Flow: ('192.168.50.140', 8080) → ('192.168.50.33', 41076)
Prediction: DDoS (conf=0.7714)
=================
```

These logs confirm that the Raspberry Pi, when operating as a real access point rather than a Mininet endpoint, provides the necessary packet timing and flow completeness required for accurate intrusion detection. The optimized TSANet model handled real-world traffic efficiently and consistently, validating the practicality of the Raspberry Pi 5 as a low-cost, edge-deployable IDS platform.

5.3 Latency and Resource Utilization

The real-time feasibility of the optimized TSANet model was evaluated by measuring latency, throughput, and hardware resource usage during continuous monitoring on the Raspberry Pi 5. Across several hours of operation, the system demonstrated stable performance without packet drops, even under high-volume attack conditions such as ICMP flooding and port-burst traffic. The average inference latency per flow ranged between 8.4 ms and 12.7 ms, allowing the device to process individual flow vectors almost instantaneously after they became idle. This corresponded to a sustained flow processing rate of approximately 70–110 flows per second, which is sufficient for small to medium IoT environments where the overall traffic load is moderate.

During peak attack bursts, CPU utilization fluctuated between 42% and 58%, illustrating that the pruning and quantization optimizations significantly reduced the computational workload of the model. Memory consumption remained stable at around 410 MB, well within the available capacity of the Raspberry Pi 5, leaving sufficient space for system processes and packet capture buffers. The end-to-end detection delay—from flow completion to alert signaling via the GPIO-connected LED—ranged from 60 ms to 110 ms. This delay includes feature extraction, normalization, inference, and alert generation, and is sufficiently low for practical real-time intrusion detection scenarios. Overall, the latency and resource utilization results confirm that the optimized TSANet model is suitable for on-device deployment in edge IoT environments, offering fast and efficient attack detection without compromising system stability.

5.4 Confusion Matrix Analysis

Table 2 shows the offline confusion matrix of the optimized TSANet model evaluated on the CICIDS-2017 test set. The matrix closely reflects the behaviour observed throughout the project,

particularly the distinction between high-volume attacks and sparse classes.

Table 2: Confusion Matrix of Optimized TSANet on CICIDS-2017

Actual / Predicted	Benign	Bot	BF	DDoS	DoS	Heartbleed	Infiltration
Benign	98.1	0.2	0.3	0.1	0.8	0.0	0.5
Bot	0.4	97.6	1.2	0.0	0.3	0.0	0.5
Brute Force	0.6	0.8	96.4	0.0	1.7	0.0	0.5
DDoS	0.0	0.0	0.0	98.9	1.1	0.0	0.0
DoS	0.4	0.1	0.3	1.5	97.3	0.1	0.3
Heartbleed	0.0	0.0	0.0	0.0	2.5	95.8	1.7
Infiltration	1.3	0.4	0.0	0.0	0.7	0.2	97.4

The confusion matrix highlights three key findings consistent with both offline and real-time observations:

- **High-volume attacks (DoS, DDoS, Brute Force) are classified with excellent accuracy.** Their strong statistical signatures and sequential patterns align well with TSANet's spatial–temporal attention layers, resulting in over 97% correct predictions.
- **Minority classes such as Heartbleed and Infiltration show occasional misclassifications.** These classes appear only rarely in CICIDS-2017, which limits the model's exposure during training. This explains why, during real-time Raspberry Pi deployment, Heartbleed generated intermittent false positives—even though its offline accuracy remained above 95%.
- **Benign traffic exhibits extremely low false-positive rates.** This matches the live deployment logs, where most day-to-day web, DNS, and HTTPS flows were confidently classified as benign.

Overall, the confusion matrix confirms that TSANet generalises well across all major attack types and that the real-time behaviour seen on the Raspberry Pi directly reflects the distributional characteristics of the original dataset.

5.5 Comparison with Baseline Models

To assess the effectiveness of the proposed TSANet architecture, its performance was compared against several widely used baseline models, including Random Forest, standalone CNN, standalone LSTM, and a CNN–LSTM hybrid. These models represent the commonly adopted families of classifiers used in intrusion detection research—tree-based learning, convolution-based spatial modeling, recurrent temporal modeling, and sequential deep hybrids.

Table 3 summarizes the comparative performance across accuracy, precision, recall, and F1-score.

Table 3: Comparison of TSANet with Baseline Models

Model	Accuracy	Precision	Recall	F1-score
Random Forest	94.2	93.7	92.5	92.8
CNN	95.6	95.1	94.8	94.9
LSTM	96.2	96.0	95.7	95.8
CNN–LSTM Hybrid	97.0	96.9	96.4	96.6
Proposed TSANet	**98.3**	**98.6**	**97.9**	**98.1**

The results clearly show that the proposed TSANet model outperforms all benchmark algorithms across all evaluated metrics. Traditional machine learning such as Random Forest performs reasonably well but struggles to capture complex temporal dependencies present in IoT traffic. CNN improves accuracy by learning spatial correlations between flow features, while LSTM captures sequential behavior more effectively. The CNN–LSTM hybrid achieves stronger performance by combining spatial and temporal learning, yet it still lacks the ability to focus selectively on the most relevant features and time steps.

TSANet surpasses these models due to its dual-attention mechanism. The spatial attention module highlights critical flow attributes, while the temporal attention module selectively emphasizes important timestamps within each flow. This allows TSANet to learn richer, context-aware representations, especially for bursty or irregular attack patterns. Furthermore, the model remains lightweight enough for Raspberry Pi deployment after pruning and quantization, making it not only more accurate but also more practical for real-world IoT intrusion detection applications.

Overall, the comparative analysis confirms that TSANet provides a significant improvement over existing baseline approaches, both in predictive performance and deployment feasibility on edge devices.

5.6 Error Analysis and Model Interpretability

A detailed error analysis was conducted to understand the limitations of the optimized TSANet model and the reasons behind specific misclassifications. Most errors were concentrated among minority classes such as Heartbleed and Infiltration, primarily due to their extremely sparse representation in the CICIDS-2017 dataset. In several cases, short-lived flows did not accumulate enough packets to form complete feature signatures, causing the model to classify them as benign because insufficient temporal evidence was available. Additionally, interpretability analysis using SHAP revealed that features such as Flow Duration, Maximum Packet Length, ACK Flag Count, and Forward IAT Mean played the most influential role in shaping model decisions. These insights indicate that while TSANet performs strongly for high-volume and burst-based attacks, rare or low-activity traffic patterns remain more challenging to classify due to intrinsic dataset limitations rather than model inefficiency.

5.7 Key Insights

The overall evaluation demonstrates that the proposed TSANet model is highly suitable for real-time intrusion detection on low-cost edge hardware. During offline training, the optimized model achieved an accuracy of 98.3% on the CICIDS-2017 dataset, confirming its strong capability to

learn both spatial and temporal attack signatures. After pruning and quantization-aware training, the model maintained the same accuracy while reducing inference latency by 43.6%, making it significantly more efficient for embedded deployment. When executed on the Raspberry Pi 5—first in a controlled Mininet environment and later as a fully functional Wi-Fi access point—the system achieved a real-time detection accuracy of approximately 96.8%. Benign flows were consistently classified with high confidence (0.80–0.95), and volumetric attacks such as DoS and DDoS were detected reliably with medium-to-high confidence scores. The Raspberry Pi sustained continuous packet capture, flow construction, and inference without packet drops, and CPU and memory usage remained stable even during periods of heavy traffic. Although rare attack types such as Heartbleed occasionally triggered false positives due to limited training samples and short-lived flow patterns, the system demonstrated robust and consistent detection behavior across several hours of live operation. Overall, these results confirm that TSANet, when combined with model compression and TorchScript optimization, can deliver practical, low-latency intrusion detection in realistic IoT network environments.

6 Conclusion and Future Work

This study developed and deployed a lightweight intrusion detection system based on the proposed Temporal–Spatial Attention Network (TSANet), designed specifically for resource-constrained IoT edge environments. By combining 1D CNNs for spatial feature extraction, LSTMs for temporal modeling, and dual attention mechanisms for selective feature emphasis, the model effectively captured complex behavioral patterns in network flows. The system was trained on the CICIDS-2017 dataset using 32 carefully engineered flow features and was further optimized through pruning and quantization-aware training to enable efficient execution on embedded hardware.

Offline experiments demonstrated that TSANet achieved an accuracy of 98.3%, outperforming several baseline deep learning models. When deployed on the Raspberry Pi 5—first in a controlled environment and later as a live wireless access point—the optimized TorchScript model consistently delivered real-time predictions with an average detection accuracy of 96.8%. The Raspberry Pi successfully handled continuous packet sniffing, flow reconstruction, feature extraction, model inference, and GPIO-based LED alerts without packet drops, confirming the practicality of TSANet as an edge IDS. Although rare attack categories like Heartbleed occasionally produced false positives, the system maintained high reliability for major attack classes and demonstrated stable behavior across prolonged operation.

Looking ahead, several promising directions can extend the capabilities of this work. One avenue is the integration of federated learning, allowing multiple Raspberry Pi nodes to collaboratively update a global IDS model without sharing raw traffic data—improving privacy and adaptability in distributed IoT networks. Continual or online learning approaches could further enhance responsiveness to evolving attack patterns that are not present in static datasets such as CICIDS-2017. Hardware acceleration using Coral TPU, NVIDIA Jetson Nano, or low-power FPGA modules could reduce latency even further and allow deeper models to run efficiently on edge devices. Finally, deploying the system in large-scale campus, smart-home, or industrial IoT testbeds would provide valuable insights into long-term performance, scalability, and resilience against real-world adversarial behaviour.

Overall, this work demonstrates that attention-enhanced deep learning models—when care-

fully optimized and paired with efficient flow reconstruction—can provide reliable, low-latency intrusion detection on low-cost edge devices. With continued advancements in on-device learning and edge-AI hardware, such systems have strong potential to become a foundational security component in future IoT and smart-city infrastructures.

Declarations

Conflict of Interest. The authors declare that they have no conflict of interest related to this research.

Ethical Approval. This study does not involve human participants, animals, or any sensitive personal data; therefore, ethical approval was not required.

Use of AI Tools. Artificial intelligence tools were used only for language refinement, editing assistance, and improving clarity. All technical content, methodology, experiments, and results were developed solely by the authors.

Funding. This research received no external funding.

Data Availability. The datasets used in this study (CICIDS2017) are publicly available and can be accessed online. Processed data generated during the study can be made available upon reasonable request.

References

[1] United Nations. Sustainable development goal 9: Industry, innovation and infrastructure. `https://sdgs.un.org/goals/goal9`, 2024. Accessed: 2025-11-11.

[2] I. Idrissi, M. Azizi, and O. Moussaoui. A lightweight optimized deep learning-based host-intrusion detection system deployed on the edge for iot. *International Journal of Computing and Digital Systems*, 11(1):210–220, 2022.

[3] T. Saba, A. Rehman, T. Sadad, H. Kolivand, and S. A. Bahaj. Anomaly-based intrusion detection system for iot networks through deep learning model. *Computers & Electrical Engineering*, 99:107818, 2022.

[4] A. Gueriani, H. Kheddar, and A. C. Mazari. Enhancing iot security with cnn and lstm-based intrusion detection systems. Technical report, University of Medea, Algeria, 2023.

[5] Y. Otoum and A. Nayak. Dl-ids: A deep learning-based intrusion detection framework for securing iot. *Transactions on Emerging Telecommunications Technologies*, 33(3):e3803, 2022.

[6] B. F. Kayode, A. S. Akintola, O. Fagbohun, E. Anaesiuba, O. Ojumah, O. Odimayo, and P. C. Amajuoyi. Temporal-spatial attention network (tsanet) for dos attack detection in network traffic. *arXiv preprint*, 2025.

[7] S. Racherla, P. Sripathi, N. Faruqui, M. A. Kabir, M. Whaiduzzaman, and S. A. Shah. Deep-ids: A real-time intrusion detector for iot nodes using deep learning. *IEEE Access*, 12:63584–63598, 2024.

[8] R. Dahiya, M. Sharma, and M. Kumar. Raspberry pi ids: A fruitful intrusion detection system for iot. In *Proc. 2nd Int. Conf. Innovative Practices in Technology and Management (ICIPTM)*, pages 1–6, Noida, India, 2024.

[9] O. O. Olakanmi and S. O. Olatunji. Optimized ensemble deep learning for real-time intrusion detection on resource-constrained raspberry pi devices. In *Procedia Computer Science*, volume 231, pages 1991–1998, 2024.

[10] R. Adrian, R. D. Mandasari, and S. Alam. Design and implementation of a machine learning-based adaptive ids on raspberry pi for smart home network security. *SITEKIN: Jurnal Sains, Teknologi dan Industri*, 22(2):334–343, 2025.

[11] S. Tsimenidis, T. Lagkas, and K. Rantos. Deep learning in iot intrusion detection. *Journal of Network and Systems Management*, 30(8):1–40, 2022.

[12] R. Morshedi, S. M. Matinkhah, and M. T. Sadeghi. Intrusion detection for iot network security with deep learning. *Journal of Artificial Intelligence and Data Mining*, 12(1):37–55, 2024.

[13] D. Gupta, N. Yadav, S. Pande, and A. Khamparia. Intrusion detection system on iot with 5g network using deep learning. *Wireless Communications and Mobile Computing*, 2022:1–13, 2022.

[14] Nathan Shone, Tran Ngoc, Vu Phai, and Qi Shi. A deep learning approach to network intrusion detection. *IEEE Transactions on Emerging Topics in Computational Intelligence*, 2(1):41–50, 2018.

[15] Md Hasan, Mohammad Islam, and Tanzim Mahmud. Lightweight deep learning models for iot intrusion detection. *Sensors*, 21:1206, 2021.

[16] Tao Tang and Yue Liu. Deep learning-based intrusion detection for iot networks. *IEEE Access*, 8:59340–59350, 2020.

[17] Ahsan Javaid, Quamar Niyaz, Weiqing Karim, et al. A deep learning framework for cyber security intrusion detection. *ICMLA*, pages 202–207, 2016.

[18] Chuanlong Yin, Yue Zhu, et al. A deep learning approach for intrusion detection using lstm. *IEEE Access*, 6:21954–21961, 2017.

[19] Ankit Roy and Prateek Trivedi. Iot intrusion detection using hybrid cnn-lstm. *Computer Communications*, 167:101–111, 2021.

[20] Jianguo Lin et al. Edge intelligence for iot security. *Future Generation Computer Systems*, 102:395–409, 2020.

[21] Yun Zhao and Jiaming Pan. Federated learning for iot intrusion detection. *IEEE IoT Journal*, 8(5):3310–3322, 2021.

[22] Mohammad Alsheikh et al. Deep reinforcement learning for iot security. *IEEE Communications Surveys & Tutorials*, 21(4):3569–3591, 2019.

[23] Xiaojun Li and Ming Wang. Attention-based networks for intrusion detection. *Neurocomputing*, 423:567–579, 2021.

[24] Lin Xu and Xia Xu. Lightweight flow-based ids for iot using statistical features. *Sensors*, 20:5822, 2020.

[25] Hoda Hindy, Ethan Bayne, et al. A taxonomy of intrusion detection in iot and cyber-physical systems. *IEEE Communications Surveys & Tutorials*, 22(1):247–291, 2020.

[26] Jonathan Barriga and Diego Chicaiza. Iot intrusion detection using lstm and autoencoders. In *IEEE SmartWorld*, pages 1–6, 2019.

[27] Faramarz Farivar et al. Distributed ids for smart homes using edge ai. *Journal of Network and Computer Applications*, 219:103658, 2023.

[28] Alsharif Abuadbba et al. Can we use machine learning securely for iot? a survey. *ACM Computing Surveys*, 53(6):1–37, 2020.

[29] Zhe Liu et al. Emerging threats and iot security challenges: A survey. *Future Internet*, 14:210, 2022.

[30] Jian He and Rui Chen. Edge computing-based ids framework for smart environments. *Journal of Systems Architecture*, 117:102139, 2021.

[31] Abdullah Derhab et al. Efficient intrusion detection for iot using deep neural models. *Computers & Security*, 113:102563, 2022.

[32] Muhammad Khan and Khaled Salah. A survey of iot security based on machine learning. *IEEE Access*, 8:167605–167652, 2020.